ShopSmart

Server — TypeScript Edition

SOLID Principles

Architecture & Refactoring Guide

Version	1.0.0
Project	ShopSmart Server
Language	TypeScript (Express + Prisma)
Classification	Internal — Architecture Team

1. Overview & Purpose

What is this document?

This document defines exactly how the five SOLID object-oriented design principles apply to the ShopSmart server codebase.

It includes: (a) the current violation in our code, (b) the rule we must follow going forward, and (c) before/after code examples drawn directly from our server files.

Every developer touching server-side code must read and follow this guide before writing or reviewing any TypeScript class, service, or module.

1.1 What Are SOLID Principles?

SOLID is an acronym coined by Robert C. Martin ('Uncle Bob') representing five fundamental principles of object-oriented software design. When consistently applied, these principles produce systems that are **easier to test**, **easier to extend**, and **far less brittle** when requirements change.

Principle	Full Name	Core Rule	Priority
S	Single Responsibility	One class — one reason to change	Critical
O	Open/Closed	Open for extension, closed for modification	Critical
L	Liskov Substitution	Subtypes must be substitutable for their base types	High
I	Interface Segregation	Clients should not depend on unused interfaces	High
D	Dependency Inversion	Depend on abstractions, not concretions	Critical

1.2 Why ShopSmart Server Needs This Now

The current server codebase was built to ship fast. It works — but as we add features, the lack of structure is creating real problems:

- Route handlers (*.routes.ts) mix HTTP plumbing, business logic, and database calls in the same function
- Adding a new feature (e.g., email notifications, payment gateway) requires editing multiple unrelated files
- Tests are hard to write because dependencies are hard-coded (prisma imported directly in routes)
- Changing email provider or database ORM would require touching dozens of files

- Code review is slow because responsibility boundaries are unclear

1.3 Current Codebase Violations At a Glance

Server Module	SOLID Violations Found	Affected Principles
auth.service.ts	Service mixes business logic, token generation, email sending	S, D
admin.routes.ts	Route handlers contain raw Prisma queries — no service layer	S, D
products.routes.ts	Route handlers double as controllers and services	S, O
orders.routes.ts	Business logic (pricing, status checks) mixed into routes	S, O, D
super-admin.routes.ts	All CRUD operations in single file, no abstraction	S, I, D
email.ts	Nodemailer tightly coupled, no interface/abstraction	D, O
prisma.ts (config)	Singleton implementation is correct but undocumented	S
errorHandler.ts	Good shape — minor coupling to Express internals	L

1.4 Architecture Vision (Post-Refactoring)

After applying SOLID, each module will follow a strict 4-layer pattern:

```
src/modules/<domain>/
  ├── <domain>.routes.ts      ← HTTP layer only (parse request, call service, send
response)
  ├── <domain>.controller.ts  ← Orchestration (calls services, handles errors)
  ├── <domain>.service.ts     ← Business logic only (pure TypeScript classes)
  ├── <domain>.validation.ts  ← Zod/Joi schemas only
  └── <domain>.types.ts       ← DTOs, interfaces, types

src/interfaces/               ← All contracts (IEmailService, IHashService …)
src/container.ts              ← Dependency injection registration
```

2. S — Single Responsibility Principle

S Single Responsibility Principle

A class (or module) should have one, and only one, reason to change.

Principle Definition

"A class should have only one reason to change." — Robert C. Martin. In our TypeScript context: each file/class should own exactly one concern. If a file changes for two different reasons (e.g., business rule change AND a database schema change), it has two responsibilities and violates SRP.

Current Issues in ShopSmart Server

- auth.service.ts handles registration, login, token generation, email sending, AND password hashing — five responsibilities in one file
- admin.routes.ts handles HTTP routing AND raw Prisma database queries (no service layer)
- orders.routes.ts contains pricing calculation, order creation, status validation, and HTTP responses mixed together
- super-admin.routes.ts is 400+ lines managing users, roles, config, feature flags, payments — all in one file
- email.ts creates the transporter AND sends emails AND formats HTML content — three separate concerns

How We Will Apply It

1. Extract every business concern into its own dedicated TypeScript class: AuthService, ProductService, OrderService, AdminService, SuperAdminService
2. Routes must ONLY handle HTTP — parse request body, call service method, return response. Zero Prisma calls in routes
3. Create an EmailService class that delegates to a transport interface (not directly to nodemailer)
4. Create a TokenService class that owns all JWT operations (signAccess, signRefresh, verify)
5. Create a HashService class that owns all hashing operations (hashPassword, comparePassword, hashToken)

Code Transformation

BEFORE — Violates Principle

```
// ❌ CURRENT: auth.service.ts does
everything
export async function register(data) {
```

AFTER — Follows Principle

```
// ✅ AFTER: Each class has ONE job
class AuthService {
  constructor(
```

```
  // hashing
  const hash = await
hashPassword(data.password);
  // db call
  const user = await
prisma.user.create({...});
  // email
  sendVerificationEmail(user.email,
token);
  // JWT
  const accessToken =
signAccessToken({...});
  return { accessToken, user };
}
```

```
    private readonly hashSvc: IHashService,
    private readonly tokenSvc:
ITokenService,
    private readonly emailSvc:
IEmailService,
    private readonly userRepo:
IUserRepository,
  ) {}
  async register(data: RegisterDTO) {
    // orchestrate — NO implementation
details
    const hash = await
this.hashSvc.hash(data.password);
    const user = await
this.userRepo.create({...hash});
    await
this.emailSvc.sendVerification(user.email);
    return
this.tokenSvc.issueTokenPair(user);
  }
}
```

Implementation Notes

Do NOT create 'god service' classes. If AuthService.ts grows beyond ~200 lines, it is violating SRP again

The test for SRP: 'If I change the email template, which file changes?' — only EmailService. If the answer is 'auth.service.ts', SRP is violated

Helper functions in utils/ (hash.ts, jwt.ts) should become injectable service classes, not bare exported functions

3. O — Open/Closed Principle

O Open/Closed Principle

Open for extension, closed for modification — add new behavior without changing existing code.

Principle Definition

"Software entities should be open for extension, but closed for modification." Software is 'open' if its behavior can be extended (by adding new code), and 'closed' if that extension doesn't require modifying existing, tested code. In TypeScript, this is achieved with interfaces and strategy/template patterns.

Current Issues in ShopSmart Server

- Adding a new payment provider (Stripe, Razorpay) requires editing the super-admin routes file — existing code must be modified
- Adding a new notification channel (SMS, push) requires editing auth.service.ts — the existing register/login functions
- Adding a new pricing rule (coupon, bulk discount) requires editing the order creation route handler
- Adding new order status transitions requires hunting and editing multiple route files
- The email.ts transporter is hardcoded to nodemailer — switching to SendGrid requires rewriting the file

How We Will Apply It

6. Define IEmailService interface — any email provider implements it (NodemailerEmailService, SendGridEmailService) without touching AuthService
7. Define INotificationService interface — add SMS or push without touching any existing service
8. Define IPaymentGateway interface — add Stripe/Razorpay by creating a new class, never by editing existing gateway code
9. Define IDiscountStrategy interface — new coupon types are new classes, not new if/else blocks
10. Define IOrderPricingStrategy — pricing rules become swappable strategies injected at runtime

Code Transformation

BEFORE — Violates Principle

```
// ❌ CURRENT: Adding SMS means editing AuthService
export async function register(data) {
  // ...
```

AFTER — Follows Principle

```
// ✅ AFTER: Add SMS without touching AuthService
interface INotificationService {
```

```
  // To add SMS: edit this function
  sendVerificationEmail(user.email,
token);
  // sendSMS(user.phone, token); // ←
modify existing code!
}
```

```
  sendVerification(to: string, token:
string): Promise<void>;
}
class AuthService {
  constructor(private notif:
INotificationService) {}
  async register(data) {
    // ... same code, works with email OR
sms
    await
this.notif.sendVerification(user.email,
token);
  }
}
// New provider = new class, zero edits to
AuthService
class SmsNotificationService implements
INotificationService {
  sendVerification(to, token) { /* twilio
*/ }
}
```

Implementation Notes

OCP does NOT mean 'never modify code'. It means protected core logic should not need to change for new features

Good rule of thumb: if you need to add an if/else or switch to existing working code to handle a new case, OCP is likely being violated

The interface definitions in src/interfaces/ are the 'closed' part — once published, they should not change (or only with versioning)

4. L — Liskov Substitution Principle

L — Liskov Substitution Principle

Subclasses must be perfectly substitutable for their base class without breaking the program.

Principle Definition

"If S is a subtype of T, then objects of type T may be replaced with objects of type S without altering any of the desirable properties of the program." — Barbara Liskov. In TypeScript: any class implementing an interface must honor the interface's full contract — not weaker preconditions, not stronger postconditions, no surprise exceptions not in the contract.

Current Issues in ShopSmart Server

- errorHandler.ts works for AppError but calling code (routes) does typeof/instanceof checks — subtypes are not cleanly substitutable
- If we add a DatabaseError or ValidationError that extends AppError with a different statusCode logic, existing error handler code must be audited
- No formal base error hierarchy exists — just AppError, which means callers treat all errors the same way
- Route handlers manually check req.user?.roleType for admin/super-admin — role-based behavior is scattered and not substitutable

How We Will Apply It

11. Create a typed error hierarchy: AppError (base) → AuthError → ValidationError → NotFoundError → ConflictError — each with correct statusCode pre-set
12. The errorHandler middleware should work identically for any AppError subclass without any instanceof checks beyond the top-level AppError check
13. Any class implementing IEmailService must honor the full contract: sendVerification and sendPasswordReset must both succeed or throw a typed EmailError
14. All service classes implementing the same interface must return the same shape — no 'extra' properties that callers might accidentally depend on

Code Transformation

BEFORE — Violates Principle

```
// ✕ CURRENT: errorHandler uses manual
type narrowing
export function errorHandler(err, req,
res, next) {
```

AFTER — Follows Principle

```
// ✅ AFTER: Subtypes substitute cleanly
class AppError extends Error {
  constructor(public statusCode: number,
```

```
  // If someone adds ConflictError(409),
  // this code must be updated manually
  const status = err instanceof AppError
    ? err.statusCode : 500;
  // New error subtypes are NOT auto-
handled
}
```

```
    message: string, public code?: string)
{
    super(message); }
}
class AuthError extends AppError {
  constructor(msg: string) { super(401,
msg, 'UNAUTHORIZED'); }
}
class ValidationError extends AppError {
  constructor(msg: string) { super(400,
msg, 'VALIDATION_ERROR'); }
}
// errorHandler works for ALL subtypes — no
changes needed
export function errorHandler(err, req, res,
next) {
  if (err instanceof AppError) {
    return
res.status(err.statusCode).json({...});
  }
  res.status(500).json({ message: 'Internal
error' });
}
```

Implementation Notes

The LSP test: replace your concrete class with any other implementation of the same interface — the calling code should not notice

If an implementation throws an exception that the interface does not declare, it violates LSP

TypeScript interfaces enforce shape but not behavior — document behavioral contracts in JSDoc on the interface methods

5. I — Interface Segregation Principle

I — Interface Segregation Principle

Clients should not be forced to depend on interfaces they do not use.

Principle Definition

"Clients should not be forced to depend on methods they do not use." — Robert C. Martin. A 'fat interface' with 20 methods forces every implementer to provide all 20, even if they need 3. Instead, define many small, focused interfaces. In TypeScript, this is clean: a class can implement multiple interfaces, and a function can declare exactly the interface it needs.

Current Issues in ShopSmart Server

- There are no interfaces at all in the current server — everything is imported directly (bare functions, direct Prisma calls)
- If we created an IUserService with 15 methods (register, login, refresh, logout, verifyEmail, forgotPassword, resetPassword, me, update, delete, list, getById, changeRole, deactivate, reactivate), every consumer would depend on all 15
- The authorize middleware needs to know user.roleType — but it imports AuthRequest which carries ALL user properties
- The admin dashboard only needs user.count and order.aggregate — but it would depend on a fat IAdminService if we create one naively

How We Will Apply It

15. Create IUserReader { findById, findByEmail } — for read-only consumers
16. Create IUserWriter { create, update, delete } — for mutation consumers
17. Create ITokenService { signAccess, signRefresh, verify } — only for token operations
18. Create IHashService { hash, compare, generateRandom } — only for hashing
19. Create IEmailService { sendVerification, sendPasswordReset } — only for email
20. Create IRoleChecker { hasRole, isAdmin, isSuperAdmin } — for authorization checks

Code Transformation

BEFORE — Violates Principle

```
// ✕ FAT interface — forces all consumers
// to depend on methods they never call
interface IUserService {
```

AFTER — Follows Principle

```
// ✓ SEGREGATED interfaces — depend on only what you need
interface IAuthService {
```

```
  register(data): Promise<User>;
  login(data): Promise<Tokens>;
  logout(token): Promise<void>;
  refresh(token): Promise<Tokens>;
  verifyEmail(token): Promise<User>;
  forgotPassword(email): Promise<void>;
  resetPassword(token, pw): Promise<void>;
  getById(id): Promise<User>;
  update(id, data): Promise<User>;
  delete(id): Promise<void>;
  // ... 10 more methods
}
```

```
  register(data: RegisterDTO):
Promise<AuthResult>;
  login(data: LoginDTO):
Promise<AuthResult>;
  logout(token: string): Promise<void>;
  refresh(token: string):
Promise<AuthResult>;
}
interface IEmailVerificationService {
  sendVerification(email: string, token:
string): Promise<void>;
  verify(token: string): Promise<User>;
}
interface IPasswordResetService {
  initReset(email: string): Promise<void>;
  reset(token: string, pw: string):
Promise<void>;
}
// AuthController depends only on
IAuthService
// VerifyController depends only on
IEmailVerificationService
```

Implementation Notes

Small interfaces are not always better — avoid going too granular (an ICreateUser with only a create() method is probably too small)

The right size for an interface: cohesive group of methods that a single client role needs

TypeScript allows 'interface extending': interface IUserManager extends IUserReader, IUserWriter { } — useful for full admin contexts

6. D — Dependency Inversion Principle

D Dependency Inversion Principle

High-level modules must not depend on low-level modules. Both should depend on abstractions.

Principle Definition

"High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details. Details should depend on abstractions." — Robert C. Martin. In practice: your business logic (AuthService) should not import PrismaClient or nodemailer directly. It should declare an interface (IUserRepository, IEmailService) and receive the concrete implementation from outside (constructor injection).

Current Issues in ShopSmart Server

- auth.service.ts imports prisma directly from config/prisma — auth business logic depends on a low-level Prisma detail
- auth.service.ts imports sendVerificationEmail directly — directly coupled to nodemailer transporter
- admin.routes.ts imports prisma directly — route handler directly depends on database layer
- All route files import prisma — 8+ files have a direct dependency on the database layer
- email.ts creates the nodemailer transporter at module load time — impossible to swap or mock in tests

How We Will Apply It

21. All services receive their dependencies through the constructor (constructor injection)
22. Create src/interfaces/ with IUserRepository, IEmailService, IHashService, ITokenService
23. Create src/repositories/ with PrismaUserRepository implementing IUserRepository
24. Create src/container.ts that wires together: new AuthService(new PrismaUserRepository(prisma), new NodemailerEmailService(transporter), ...)
25. Route files depend on interfaces, never on concrete implementations

Code Transformation

BEFORE — Violates Principle

```
// ✕ CURRENT: auth.service.ts violates
DIP
import { prisma } from
'../../config/prisma'; // ← low-level!
```

AFTER — Follows Principle

```
// ✅ AFTER: AuthService depends on
abstractions only
// src/interfaces/IUserRepository.ts
interface IUserRepository {
```

```
import { sendVerificationEmail } from
'../../utils/email'; // ← low-level!

export async function register(data) {
  // business logic tightly coupled to:
  // - Prisma ORM (database)
  // - nodemailer (email)
  // Can't test without a real DB and SMTP
server!
  await prisma.user.create({...});
  sendVerificationEmail(data.email,
token);
}
```

```
  findByEmail(email: string): Promise<User
| null>;
  create(data: CreateUserDTO):
Promise<User>;
}
// src/modules/auth/auth.service.ts
class AuthService {
  constructor(
    private readonly users:
IUserRepository,
    private readonly email: IEmailService,
    private readonly tokens: ITokenService,
  ) {}
  async register(data: RegisterDTO) {
    // Only abstractions — fully testable
with mocks!
    const user = await
this.users.create(data);
    await
this.email.sendVerification(user.email);
    return
this.tokens.issueTokenPair(user);
  }
}
```

Implementation Notes

DIP is the most impactful principle for testability — once DIP is applied, unit tests can inject mock repositories and never touch a real database

Use constructor injection (not property injection, not service locator) — it makes dependencies explicit and visible

The src/container.ts file is the ONLY place where concrete classes are instantiated — everywhere else works with interfaces

7. Implementation Roadmap

How to use this roadmap

Work through the tasks below in priority order: Critical first, then High, then Medium.
Each task corresponds directly to a SOLID principle. The 'P' column shows which principle.
Do NOT mix refactoring commits with feature commits — each refactoring task should be its own PR.

P	Task	Files Affected	Priority
S	Create IEmailService interface in src/interfaces/	auth, email	Critical
S	Extract AuthService class from auth.service.ts (functions → class methods)	auth	Critical
S	Create ProductService class (extract from products.routes.ts)	products	Critical
S	Create OrderService class (extract from orders.routes.ts)	orders	Critical
S	Create AdminService class (extract from admin.routes.ts)	admin	Critical
S	Create SuperAdminService class (extract from super-admin.routes.ts)	super-admin	Critical
O	Create INotificationStrategy interface for email/SMS/push	email, notifications	High
O	Create IOrderPricingStrategy interface for pricing variations	orders	High
O	Create IAuthProvider interface for token strategies	auth	High
L	Add ErrorHandler base class with typed subclasses (ValidationError, AuthError)	middleware	Medium
L	Ensure all service classes are substitutable (no hardcoded type checks in routes)	all modules	High
I	Create focused interfaces: IUserReader, IUserWriter (vs fat IUserService)	user, users	High
I	Create ITokenService for JWT operations only	auth, jwt	Medium
I	Create IHashService for hashing operations only	auth, hash	Medium
D	Wire NodemailerEmailService via dependency injection (not direct import)	email	Critical
D	Wire PrismaClient via DI container or service constructor injection	all services	Critical
D	Create src/container.ts — DI registration file	root	Critical

7.1 Recommended File Structure After Refactoring

```
server/src/
├── interfaces/              ← NEW: all contracts live here
│   ├── IEmailService.ts
│   ├── IHashService.ts
│   ├── ITokenService.ts
│   ├── IUserRepository.ts
│   ├── IProductRepository.ts
│   ├── IOrderRepository.ts
│   └── INotificationService.ts
├── repositories/            ← NEW: Prisma implementations
│   ├── PrismaUserRepository.ts
│   ├── PrismaProductRepository.ts
│   └── PrismaOrderRepository.ts
├── services/                ← NEW: cross-cutting services
│   ├── NodemailerEmailService.ts
│   ├── BcryptHashService.ts
│   └── JwtTokenService.ts
├── container.ts             ← NEW: DI wiring
├── modules/
│   ├── auth/
│   │   ├── auth.routes.ts    ← HTTP only (no Prisma)
│   │   ├── auth.controller.ts
│   │   ├── auth.service.ts   ← Class, not functions
│   │   └── auth.validation.ts
│   ├── products/  (same structure)
│   ├── orders/    (same structure)
│   └── ...        (same structure)
└── config/ middleware/ utils/ (unchanged)
```

7.2 Testing Strategy (Enabled by SOLID)

Once DIP is applied via constructor injection, unit testing becomes trivial:

```
// Unit test for AuthService — NO database, NO SMTP needed
describe('AuthService.register', () => {
  it('sends verification email after creating user', async () => {
    const mockUserRepo: IUserRepository = {
      findByEmail: jest.fn().mockResolvedValue(null),
      create: jest.fn().mockResolvedValue({ id: '1', email: 'a@b.com' }),
    };
    const mockEmail: IEmailService = {
      sendVerification: jest.fn().mockResolvedValue(undefined),
      sendPasswordReset: jest.fn(),
    };
    const svc = new AuthService(mockUserRepo, mockEmail, mockTokenService);
    await svc.register({ email: 'a@b.com', password: 'pass123', fullName: 'Test'
});
    expect(mockEmail.sendVerification).toHaveBeenCalledWith('a@b.com',
expect.any(String));
  });
});
```

7.3 Quick Reference: The Five Questions

Before writing any new TypeScript class or function in the server, answer these five questions:

#	Question to Ask	If No → Action Required
S	*Does this class/file have exactly ONE reason to change?*	Split it. Extract the additional concern into its own class.
O	*Can I add this new feature WITHOUT editing existing working code?*	Define an interface. The new feature becomes a new class implementing it.
L	*Can I replace my concrete class with another implementation of the same interface without the caller knowing?*	Review the interface contract. Add missing guarantees. Remove hidden exceptions.
I	*Does my interface only include methods this specific client actually needs?*	Split the interface. Create separate focused interfaces.
D	*Does my class/function import any concrete class (Prisma, nodemailer, bcrypt)?*	Replace the import with an interface. Inject the concrete via constructor.

End of SOLID Principles Documentation — ShopSmart Server

Next: Design Patterns Documentation (Part 2)